IV. Evaluation

本节从性能、机制消融、配置敏感性和硬件开销四个方面评估 PIVOT。具体而言，我们首先验证动态 Bank 映射能否缓解 MoE 执行中的片上访存停顿；随后分析静态预取、动态映射和协同预取之间的差异；进一步考察专家数量、Token 数量、Top-k 和路由分布变化下的鲁棒性；最后通过逻辑综合评估 PIVOT 的面积和功耗开销。

A. Methodology

我们基于 SCALE-Sim v3 扩展了一个周期索引的 Bank 冲突感知事件模型，用于刻画 MoE 执行期间 IA、Weight、OA 和 ACC 请求在片上 Bank 中的排队、服务与冲突行为。该模型使用 SCALE-Sim 生成计算执行区间，并显式维护 Bank 端口、请求队列、虚拟对象映射和 Weight Chunk 生命周期。控制通路握手、详细 DRAM 时序以及多 GPU NoC 传输被抽象为统一的延迟与带宽模型，并对所有对比方案采用相同的资源和时序假设。

实验配置见表 1，默认加速器配置包含 30 个物理 Bank。每个 Bank 包含 128 个 128-bit 存储条目，对应 2 KB 容量和 16 B/cycle 带宽，片上总容量为 60KB，总聚合带宽为 480 B/cycle。每个 Bank 配置一个访问端口和深度为 32 的请求缓冲区。IA、Weight 和 OA 使用 INT8，ACC 使用 INT32。普通写入按照单次 Bank 传输进行建模，而 ACC 更新包含读、加法和写回三个阶段。

我们选取 HMoE、Mixtral、MoDSE 和 Switch Transformer 四种缩放 MoE 工作负载。默认配置包含 256 个 Token、8 个专家和 Top-1 路由，并同时覆盖同构与异构专家结构。需要说明的是，这些工作负载用于保持不同 MoE 结构和访存行为的相对特征，而不是对公开模型完整层数进行端到端复现。

本文比较以下四类方案：

- Static：采用固定 IA、Weight、OA 和 ACC Bank 所有权，不启用预取；
- Dynamic：采用动态 Bank 映射，但不启用预取；
- Dynamic+PF：采用动态 Bank 映射和固定 Window/Chunk 的非协同预取；
- PIVOT：采用动态映射、Bank 压力感知放置、窗口控制和在线保护机制。

所有方案使用相同的计算阵列、Bank 数量、总容量、带宽、端口数量和工作负载。本文使用总执行周期和 Memory Stall Cycles 作为主要性能指标，并通过

$$\text{Reduction} = \frac{T_{\text{base}} - T_{\text{scheme}}}{T_{\text{base}}} \times 100\%$$

计算相对于基线的周期下降比例。跨模型的平均下降比例采用算术平均。

敏感性实验进一步改变专家数量、Token 数量、Top-k、路由倾斜程度和随机路由种子。

预取实验使用 W∈{0,1,2,4,8,16,32,64}的预取窗口，以及 C∈{1,2,4,8} tiles 的 Chunk 粒度。在当前 MoDSE 置中，一个 Tile 对应 2 KB。

Item	Configuration
Simulator	SCALE-Sim v3 with a bank-conflict-aware PIVOT extension
Workloads	HMoE, Mixtral, MoDSE, and Switch Transformer
Default MoE setting	256 tokens, 8 experts, Top-1 routing
Data precision	IA / Weight / OA: INT8; ACC: INT32
On-chip memory	30 Banks, 2 KB per Bank, 60 KB total
Bank configuration	16 B/cycle/Bank, 1 port/Bank, queue depth 32
Prefetch parameters	Window {0, 1, 2, 4, 8, 16, 32, 64}; Chunk {1, 2, 4, 8} tiles
Sensitivity parameters	Experts {4, 8, 16}; Tokens {32, 128, 256, 512}; Top-k {1, 2}
Evaluated schemes	Static, Dynamic, Dynamic+PF, and PIVOT

表 1 Experimental Setup and Evaluated Configurations

B. Overall Performance and Bank-Level Behavior

我们首先单独评估动态 Bank 映射带来的性能收益，以排除预取机制的影响。如图 4(a)所示，Dynamic 在四个 MoE 工作负载上均显著降低了 Memory Stall Cycles。HMoE 的访存停顿由 84,719 cycles 降低至 61,435 cycles，下降 27.48%；Mixtral、MoDSE 和 Switch Transformer 的访存停顿分别下降 24.96%、22.48%和 23.25%。四个工作负载的平均 Memory Stall 降低 24.54%。

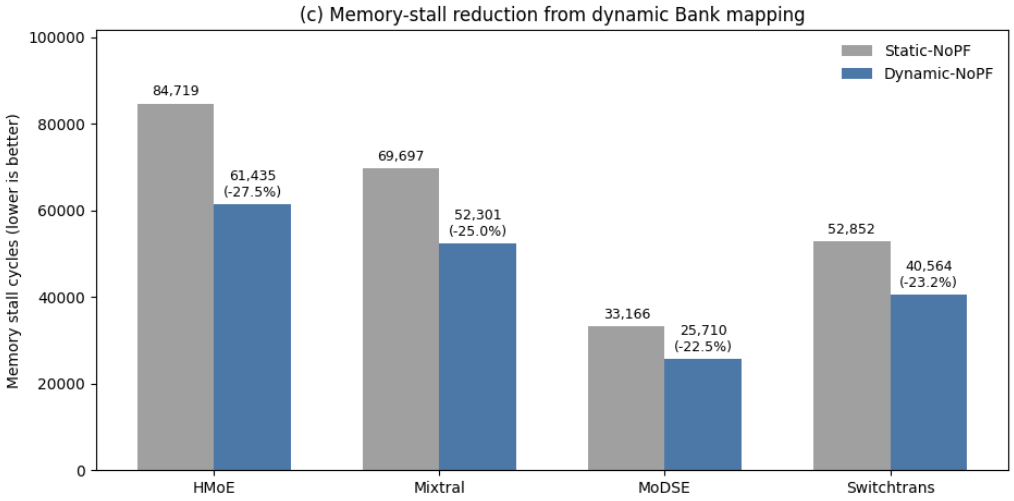


图 4a Memory-stall cycles under static and dynamic bank mappings.

如图 4(b)所示，访存停顿的降低进一步转化为端到端执行周期收益。Dynamic 相比 Static

在 HMoE、Mixtral、MoDSE 和 Switch Transformer 上分别降低总周期 10.11%、8.35%、6.12% 和 7.30%，平均降低 7.97%。其中，HMoE 获得最高收益，而 MoDSE 的收益相对较低，但所有工作负载均未出现性能回退。

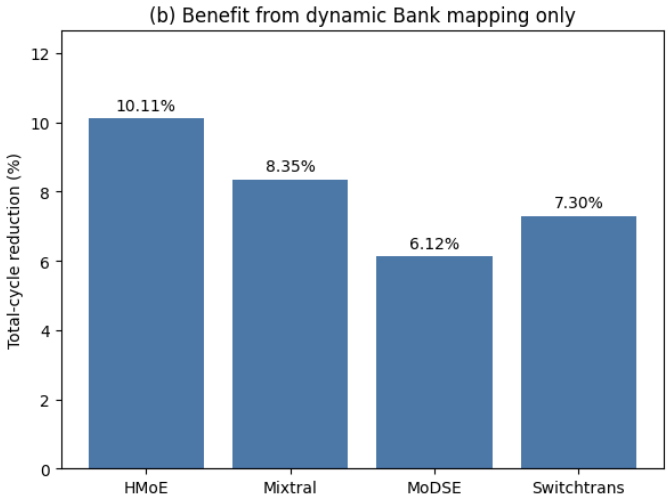


图 4b Total-cycle reduction achieved by dynamic bank mapping.

值得注意的是，Memory Stall 的下降幅度明显高于总周期下降幅度。这是因为动态 Bank 映射主要优化访存等待部分，而计算周期本身保持不变。该结果表明，固定 Bank 所有权确实会在 MoE 执行过程中产生局部资源拥塞；通过根据阶段级需求重新分配物理 Bank，可以减少请求排队并提高 Bank 资源的有效利用率。

不过，动态映射本身并不能隐藏 Weight 数据搬运延迟，也无法决定未来数据应当提前多远进入 SRAM。因此，Dynamic 相对于 Static 的收益构成了后续协同预取能够继续优化的基础。

C. Ablation Study

为分析动态 Bank 映射与预取机制之间的关系，我们进一步对 Static、Dynamic、Dynamic+PF 和完整 PIVOT 进行机制级消融。这里的实验重点不是严格逐个硬件模块关闭，而是比较静态放置、动态放置、固定预取和协同预取四种策略组合。

如图 5(a)所示，在静态 Bank 映射下，固定预取对 Window 和 Chunk 参数高度敏感。适中的预取窗口可以通过计算—传输重叠降低执行周期，但当 Window 或 Chunk 过大时，预取数据会过早进入片上 SRAM，并持续占用有限的 Bank 容量、端口和请求队列，最终造成明显性能回退。这说明“更早发起”和“搬运更多数据”并不必然带来更高收益。

如图 5(b)所示，动态 Bank 映射扩大了固定预取的有效配置区域。由于预取 Chunk 可以被放入更合适的空闲 Bank，一部分静态映射下的热点和容量冲突得到缓解。然而，Dynamic+PF 在激进的 Window/Chunk 配置下仍然会出现性能下降。其根本原因是动态映射只能改善数据放置，无法独立解决预取及时性、数据驻留时间以及当前计算与未来预取之间

的资源竞争。

PIVOT 在动态映射基础上进一步联合考虑目标 Bank、Chunk 驻留压力和预取窗口。如图 5(c)所示，PIVOT 在大多数 Window/Chunk 配置下均保持正收益，相对于 Static-NoPF 的总周期最高降低 25.25%。与只使用固定预取的方案相比，PIVOT 能够在资源压力增大时缩小、延迟或保护预取决策，从而避免激进参数直接演化为严重性能回退。

如图 5(d)所示，即使与每个 Window/Chunk 配置下的最强传统方案比较，PIVOT 仍然能够获得最高 20.16%的额外周期下降。这一结果表明，PIVOT 的收益并非简单来自动态映射或预取中的任意单一机制，而是来自数据放置与预取决策之间的协同：动态映射为未来 Chunk 提供可调整的物理位置，而 Bank 感知的预取策略则避免这些 Chunk 与当前计算流争用同一组资源。

需要指出的是，图中的 PIVOT 结果包含在线保护机制。当当前预取候选无法优于已有安全方案时，控制器可以退回非激进配置。因此，无回退现象应归因于完整的协同策略与保护机制，而不能简单解释为未经保护的原始预取策略天然鲁棒。

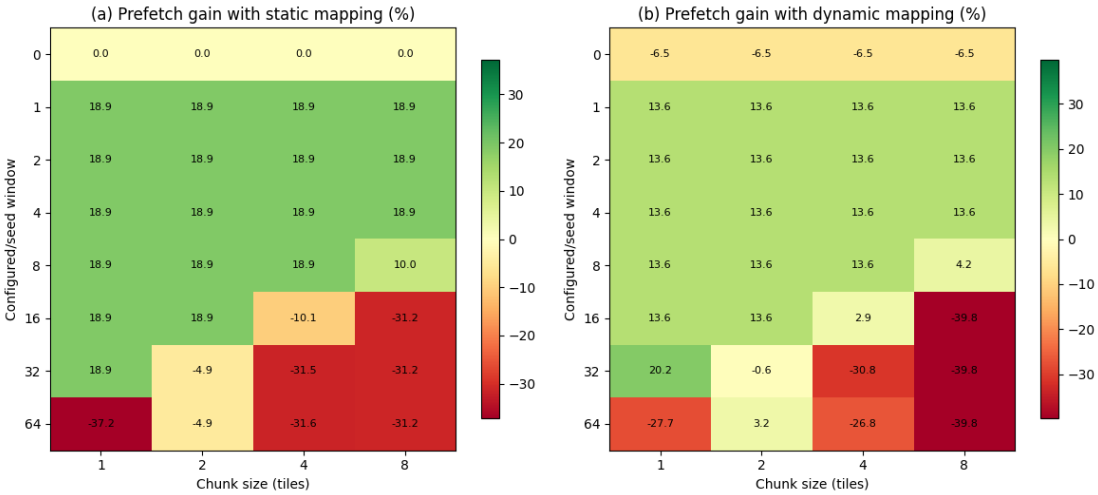
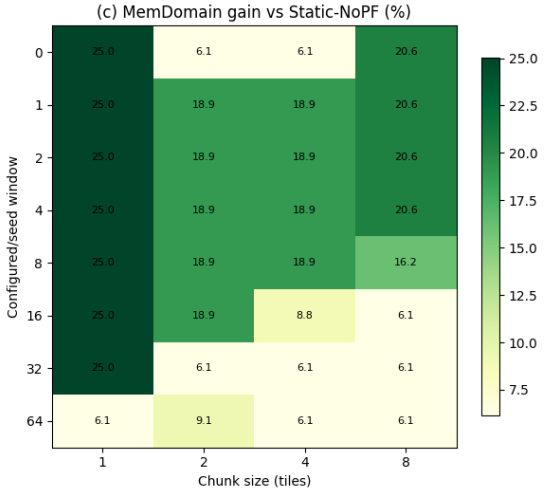


图 5a Fixed prefetching under static bank mapping .图 5b Fixed prefetching under dynamic bank mapping.



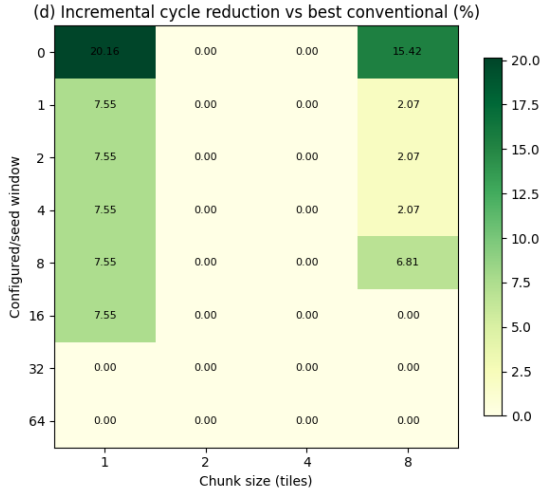


图 5c PIVOT's cycle reduction over Static-NoPF.　　图 5d PIVOT's incremental improvement over the strongest

D. Sensitivity Study

模型规模与路由复制敏感性：我们首先改变专家数量、Token 数量和 Top-k，分析 PIVOT 是否只对默认 MoE 配置有效。

如图 6(a)所示，随着专家数量由 4 增加到 16，所有方案的执行周期均有所增加，但 Static 的周期增长速度通常更快，使 PIVOT 在专家数量较多时表现出更明显的优势。专家数量增加会引入更多不同尺寸和生命周期的数据对象，并提高 IA、Weight、OA 与 ACC 流量随时间变化的程度。固定 Bank 所有权难以同时适配这些阶段，而动态映射能够根据活跃专家需求重新利用空闲 Bank。

如图 6(b)所示，当 Token 数量由 32 增加到 512 时，专家 GEMM 的 M 维度以及 IA、OA 和 ACC 流量同步增加。尽管更大的 Token 数显著提高了片上访存压力，PIVOT 在四个工作负载上仍保持稳定的周期收益。这说明其优势并不依赖于某一个固定的 Token 批量。

如图 6(c)所示，Top-k 从 1 增加到 2 后，每个 Token 被复制并发送至更多专家，因而同时增加 IA/OA 流量、被激活的 Weight Chunk 数量和 ACC 写回压力。Static 和仅动态映射方案的执行周期均明显提高，而 PIVOT 仍然能够通过动态放置和预取协同保持收益。

综合专家数量、Token 数量和 Top-k 的实验结果，PIVOT 能够适应由模型规模和路由复制产生的访存流变化，而非仅针对默认的 8-expert、256-token、Top-1 配置进行优化。

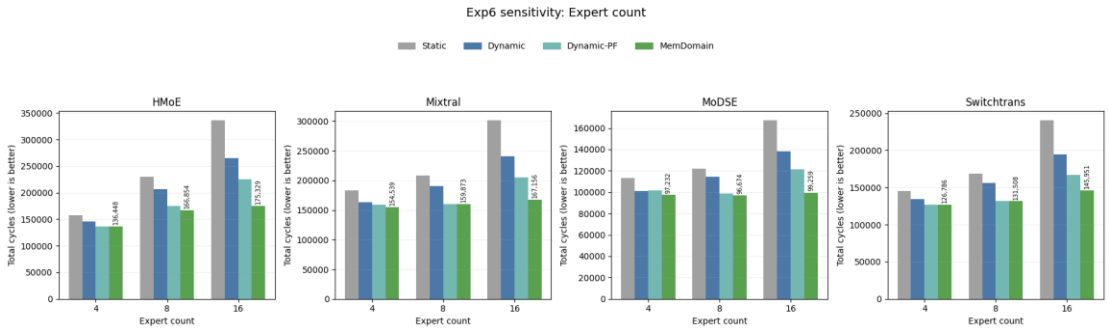


图 6a Expert count sensitivity；

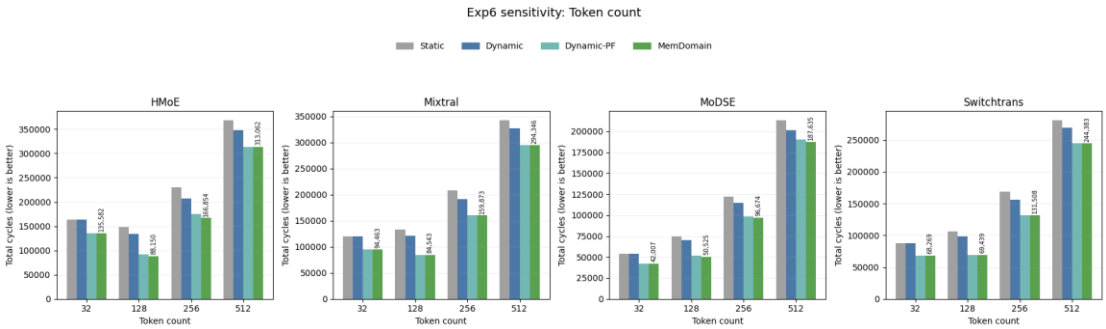


图 6B Token count sensitivity；

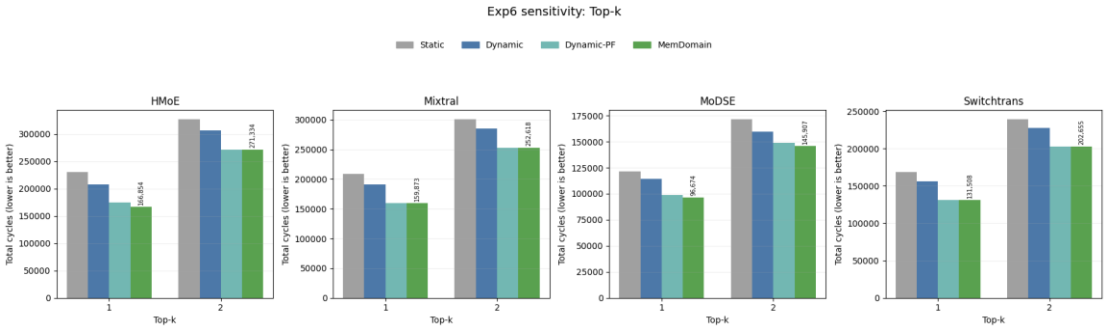


图 6c Top-k sensitivity；

路由分布敏感性：MoE 路由具有输入相关性，即使模型结构保持不变，不同输入或随机种子也会改变专家负载分布。

如图 6(d)所示，在 balanced、light-skew 和 high-skew 三种路由分布下，PIVOT 在 HMoE、Mixtral、MoDSE 和 Switch Transformer 上均保持良好收益。当路由分布更加倾斜时，少数专家接收更多 Token，并在对应时间区间形成更集中的 IA、Weight 和 ACC 压力。固定映射无法及时调整这些专家所拥有的 Bank 数，而 PIVOT 可以将当前空闲资源重新分配给热点数据对象。

如图 6(e)所示，不同 routing seed 会进一步改变各专家的实际 Token 分布，使 Static 的执行周期产生明显波动。相比之下，PIVOT 在多个随机种子下均保持较稳定的性能。这说明动态映射和预取协同能够部分吸收路由随机性造成的 Bank 需求扰动。

在全部 96 个模型与路由配置中，带保护机制的 PIVOT 相对于 Static-NoPF 均未出现性能回退，总周期下降范围为 12.09%–47.94%。因此，PIVOT 的收益不仅存在于单个工作负载或固定路由分布，而是在专家规模、Token 数、Top-k 和路由倾斜发生变化时仍具有较好的鲁棒性。需要强调的是，该无回退结果包含在线保护机制，不能被解释为未经保护的原始策略在所有配置下均不会退化。

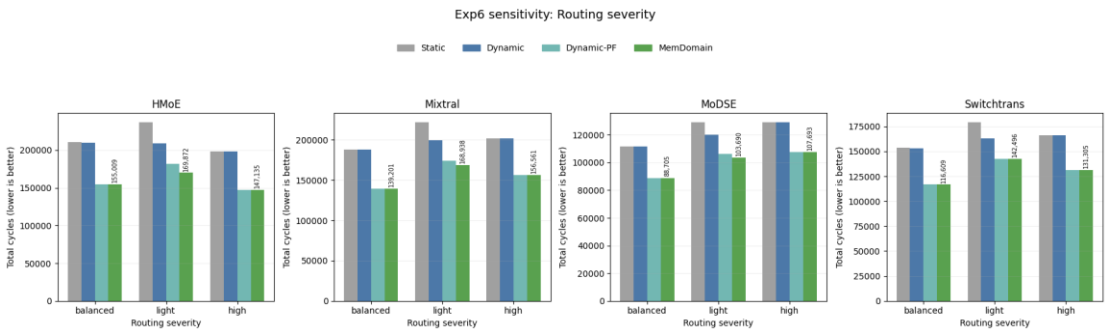


图 6d Routing severity sensitivity；

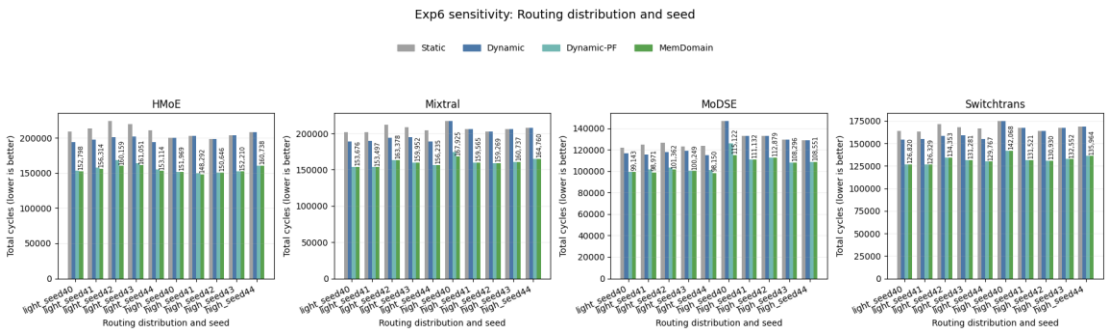


图 6e Routing seed sensitivity。

E. Hardware Overhead and Synthesis Results

我们使用 Synopsys Design Compiler，在相同工艺库和时序约束下对静态基线和 PIVOT 进行逻辑综合。两种设计采用相同的存储规模和面积统计口径，因此表中的差异主要来自统一访问路径、动态映射、Bank 分配以及预取控制相关逻辑。

如表 II 所示，静态基线的总标准单元面积为 47,353.355 μm2，PIVOT 的总标准单元面积为 53,877.754 μm2，面积增加 13.78%。其中，组合逻辑面积由 38,257.191 μm2 增加至 44,108.919 μm2，增幅为 15.30%；时序逻辑面积由 9,096.164 μm2 增加至 9,768.836 μm2，增幅为 7.40%。组合逻辑增幅高于时序逻辑，说明主要额外开销来自映射查询、候选 Bank 选择和统一请求控制，而不是大规模状态存储。

PIVOT 的动态功耗由 3.667 mW 增加至 4.113 mW，增幅为 12.16%；泄漏功耗由 150.118 μW 增加至 169.189 μW，增幅为 12.70%。总体功耗由 3.818 mW 增加至 4.281 mW，对应 12.14% 的开销。

进一步的模块级分析表明，Backend 是主要硬件开销来源，占 PIVOT 总面积的 61.13% 和总功耗的 60.90%。这是因为虚拟—物理 Bank 映射、请求仲裁、AccPipe 以及物理 Bank 访问控制主要集中于后端。相比之下，Frontend 和 Midend 主要承担指令解析与通道绑定，其资源占比较低。

总体而言，PIVOT 以 13.78%的标准单元面积开销和 12.14%的功耗开销，换取了显著的执

行周期和 Memory Stall 降低，表明动态 Bank 虚拟化与协同预取能够在可控硬件成本下实现。

指标	静态基线架构	本文研究架构	增加比例
Total Cell Area/μm²	47353.355	53877.754	13.78%
Combinational Area/μm²	38257.191	44108.919	15.30%
Sequential Area/μm²	9096.164	9768.836	7.40%
Dynamic Power/mW	3.667	4.113	12.16%
Leakage Power/μW	150.118	169.189	12.70%
Total Power/mW	3.818	4.281	12.14%

表 2 synthesis Results of the Static Baseline and PIVOT